

Dual-Core Machine Learning Accelerator for Attention Mechanism

Group 3

Team of six — names withheld in this public copy

*Deliverable hosted on the course server
(path withheld in this public copy)*

1 Introduction

This report presents our ECE260B final project on a dual-core accelerator for the attention mechanism. Starting from the provided single-core matrix-multiplication framework, we extended the design step by step to include normalization, backend implementation, and dual-core operation across different clock domains. We also explored several optimization and sparsity-aware ideas, and checked the design behavior through simulation at each stage.

2 Single Core RTL & Verification

In Step 1, we implemented the single-core 8×8 query-key multiplication, where each query and key vector has eight signed 8-bit elements. In our design, `qmем` and `kmем` provide the input data, the MAC array performs the dot-product computation, the output FIFO buffers the intermediate results, and `psum_mem` stores the final outputs.

For verification, we used the provided testbench with `qdata.txt` and `kdata.txt`. The testbench first prints the expected multiplication results, and the hardware outputs are then checked using the waveform and the “Memory write to PSUM mem” log. As shown in Fig. 1, the values written into PSUM match the expected results from the testbench. Fig. 2 shows the corresponding single-core waveform. Together, these results confirm that the single-core RTL behaves correctly.

```
VCD info: dumpfile fullchip_tb.vcd opened for output.
#### Q data txt reading ####
#### K data txt reading ####
#### Estimated multiplication result ####
prd @cycle 0: 0005dfffecf90005efffd0fffd3fffc1fffaa
prd @cycle 1: 00034fffebfbf700018fffbfffdfffd7fffe9
prd @cycle 2: 000260001b000320003ffffe400015ffffdffc9
prd @cycle 3: 00021fffb6000110007dfffdbfffeffff5fffc8
prd @cycle 4: 00050ffff6ffff60000dff76ffdbfffa6fffc8
prd @cycle 5: 0002a000010001a000440000700007ffff8fffd3
prd @cycle 6: fffed0003000063ffff9fffff000170002efffec
prd @cycle 7: ffffc10002900080fffd2fffff0004e0004a00025
#### Qmem writing ####
#### Kmem writing ####
#### K data loading to processor ####
#### execute ####
#### move ofifo to pmem ####
Memory write to PSUM mem add 00 0005dfffecf90005efffd0fffd3fffc1fffaa
Memory write to PSUM mem add 01 00034fffebfbf700018fffbfffdfffd7fffe9
Memory write to PSUM mem add 02 000260001b000320003ffffe400015ffffdffc9
Memory write to PSUM mem add 03 00021fffb6000110007dfffdbfffeffff5fffc8
Memory write to PSUM mem add 04 00050ffff6ffff60000dff76ffdbfffa6fffc8
Memory write to PSUM mem add 05 0002a000010001a000440000700007ffff8fffd3
Memory write to PSUM mem add 06 fffed0003000063ffff9fffff000170002efffec
Memory write to PSUM mem add 07 ffffc10002900080fffd2fffff0004e0004a00025
```

Figure 1: Expected multiplication results and pmem write log from simulation.

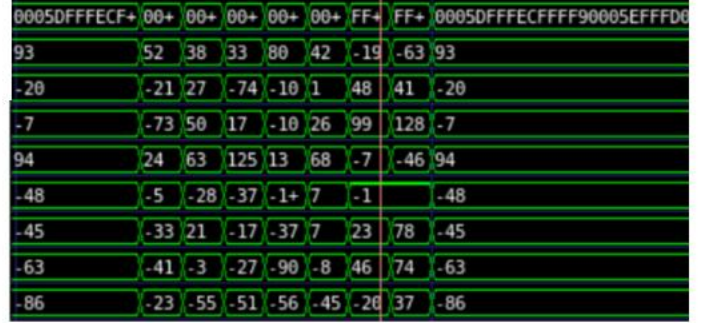


Figure 2: Single-core output waveform from simulation.

3 Output Normalization

In Step 2, the multiplication results stored in `psum_mem` are normalized row by row. For each row, the normalization block reads the eight partial-sum values, takes their absolute values, and accumulates them to form the row sum. In our implementation, this operation is handled inside the `sfp_row` block. To preserve enough precision in fixed-point hardware, each absolute-value term is shifted left by 8 bits before division. The normalized output is computed as

$$\tilde{y}_j = \frac{|y_j| \times 2^8}{\sum_{k=0}^7 |y_k|},$$

where y_j denotes one element in the selected PSUM row. For verification, the testbench first computes the expected normalized outputs from the multiplication results and prints them cycle by cycle. The hardware results are then checked using both the normalization waveform and the values written back to memory. Fig. 3 shows the normalized output waveform. Fig. 4 shows the expected normalization results from the testbench, and Fig. 5 shows the corresponding hardware results. The agreement between them confirms the correctness of the normalization stage.

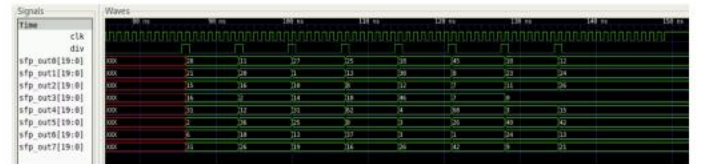


Figure 3: Normalized output waveform from behavioral simulation.

4 Single Core Synthesis & PNR

In this stage, the completed single-core RTL was synthesized and then placed-and-routed under a 1GHz target.

[illegible]

Figure 4: Expected normalization results generated by the testbench.

[illegible]

Figure 5: Hardware normalization results written back to memory.

The main implementation results are summarized in Table 1. Although the post-route timing slack remains slightly negative, the single-core design completes the backend flow successfully and provides the baseline for the later hierarchical and dual-core implementations. The final flat single-core PNR layout is shown in Fig. 6, while the detailed timing, power, and DRC reports are included in the appendix.

Metric	Value
Area	119724.48 μm^2
Total Power	60.38 mW
Post-route WNS	-0.021 ns

Table 1: Main implementation results of the flat single-core design.

5 Single-Core Hierarchical Synthesis

In this stage, the single-core design was implemented with a hierarchical backend flow. The SRAM-related blocks were handled separately and then integrated into the full core as hierarchical blocks. Compared with the flat single-core implementation, this flow gives better control over macro placement, pin assignment, and routing organization, and it also provides a cleaner basis for the later dual-core integration.

Table 2 summarizes the main implementation results of the hierarchical single-core design. The corresponding hierarchical PNR layout is shown in Fig. 7, while detailed post-route timing and power reports are included in the appendix.

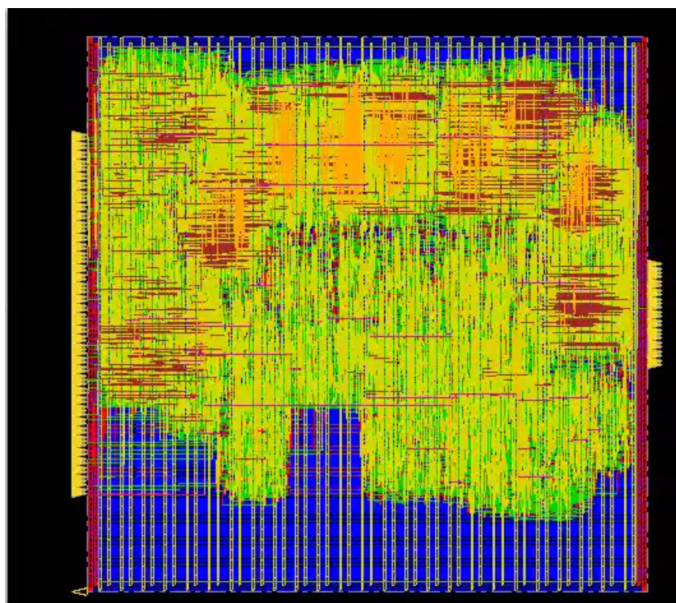


Figure 6: Place-and-route layout of the flat single-core design.

Metric	Value
Area	287419.32 μm^2
Total Power	685.38 mW
Post-route WNS	-0.831 ns

Table 2: Main implementation results of the hierarchical single-core design.

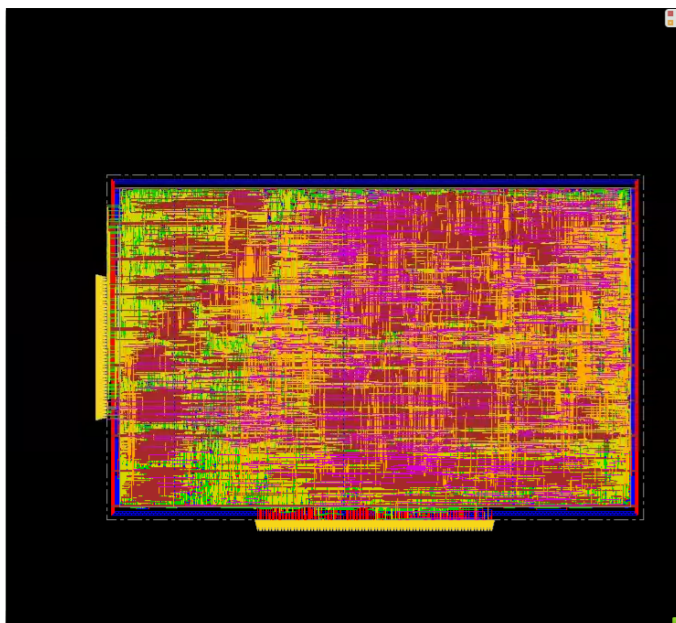


Figure 7: Hierarchical place-and-route layout of the single-core design.

6 Dual Core RTL Synth & PNR

To support 16-vector processing, the single-core architecture was extended to a dual-core design. Each core processes eight vectors, and the normalization denominator is formed by combining the row sums from both cores. Because the two cores run in different clock domains, an asynchronous interface was used to exchange the required normalization signals. Alignment logic was also added so that the merged denominator and the corresponding data arrive at the division stage in the correct cycle.

After completing the dual-core integration, the full design was synthesized and placed-and-routed using a hierarchical flow. The main implementation results are summarized in Table 3. Although the timing slack remains slightly negative, the dual-core backend flow was completed successfully. The final dual-core PNR layout is shown in Fig. 8, while the detailed power and timing reports are included in the appendix.

Metric	Value
Area	283408.92 μm^2
Total Power	580.99 mW
Post-route WNS	-0.070 ns

Table 3: Main implementation results of the dual-core design.

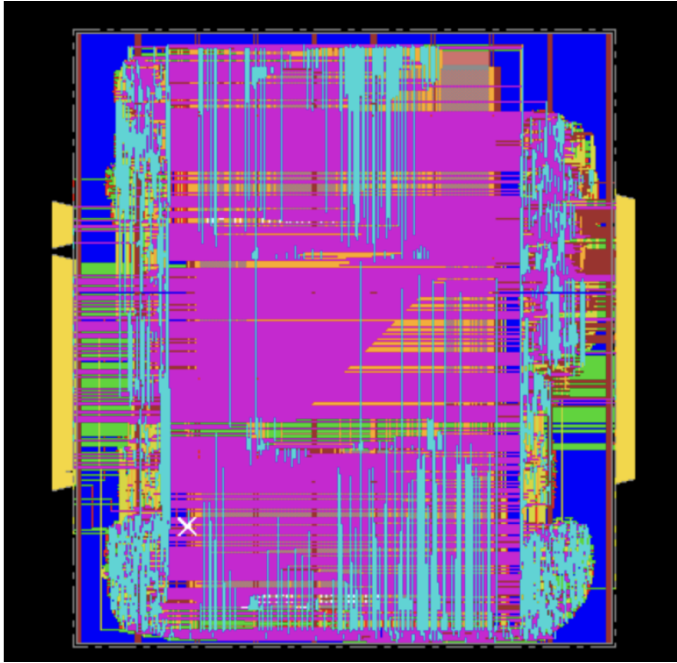


Figure 8: Final place-and-route layout of the dual-core design.

7 Functional Verification and Gate-Level Simulation

The design was verified by comparing the simulated outputs with the expected results generated by the testbench. For the normalization stage, the reference outputs were computed from `qdata.txt` and `kdata_core0/1.txt`. For the norm-times-value stage, the expected results were generated from `vdata.txt` and `norm_core0/1.txt`. The hard-

ware outputs were then checked against these reference values using the waveform.

Fig. 9 shows the verification waveform of the completed design. The `sfp_out` signals produce stable output values cycle by cycle, and the observed behavior matches the expected results from the testbench. This confirms the functional correctness of the final implemented stage.

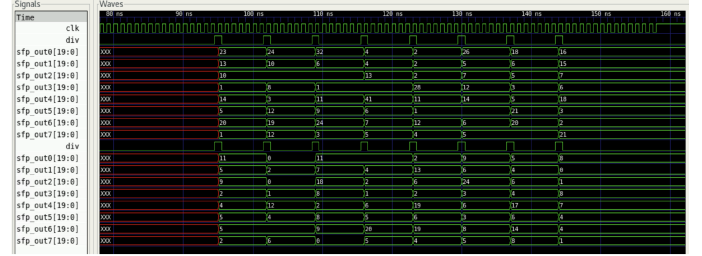


Figure 9: Final functional verification waveform of the completed design.

8 Optimization

In Step 5, we focused on reducing timing pressure and making the dual-core design more robust. The main bottlenecks appeared in the MAC datapath, the normalization path, the buffering structure, and the communication path between the two cores. To address these issues, we introduced several optimizations, including deeper MAC pipelining, a more structured accumulation path, safer clock-control logic, improved buffering, and better signal alignment in the dual-core flow.

8.1 Optimization Goals

The main goal of this step was to improve timing while keeping the dual-core computation correct. In the earlier version, long combinational paths were observed in the multiply-accumulate logic and in the normalization-related dataflow. These paths increased timing pressure and made dual-core integration harder. The optimized design therefore aimed to break these paths into smaller stages, improve coordination between stages, and align the cross-core signals more carefully.

8.2 Pipelined MAC Path

One major optimization was applied to the MAC datapath. In the original version, multiplication and accumulation were connected through a relatively long arithmetic path. In the optimized version, additional pipeline stages were inserted so that multiplication results are first registered and then accumulated in later stages. The accumulation structure was also reorganized into a more balanced adder tree instead of a long linear reduction path. These changes shorten the critical path and make the MAC block easier to integrate into the full dual-core design.

8.3 Clocking and Power Optimization

To reduce unnecessary switching activity, the optimized design also includes clock-control logic and selective computation. Clock gating was applied with controlled enable conditions so that inactive logic does not toggle unnecessarily. In addition, invalid or zero-valued inputs can be prevented from propagating through parts of the MAC path. These

changes mainly target dynamic activity in the datapath while keeping the timing behavior manageable.

8.4 FIFO and Buffering Adjustment

The FIFO structure was also adjusted in the optimized version. A deeper FIFO was added to better match the extra pipeline stages and the longer dataflow latency. The mux path was also widened to support the larger buffering structure and the related output selection. These changes help keep data movement more stable across stages and reduce timing pressure caused by the added latency.

8.5 Asynchronous Interface Handling

Because the two cores operate in different clock domains, reliable inter-core communication was also part of the optimization effort. In our implementation, the cross-core path uses a 4-phase handshake interface together with synchronizing logic to transfer normalization-related information safely. This is important because the merged sum must arrive correctly even when the two sides are not perfectly aligned in time.

8.6 Parallelism and System-Level Optimization

The optimized design also improves system-level coordination by overlapping computation, buffering, and cross-core communication as much as possible. Each core continues to process its own portion of the workload in parallel, while the surrounding logic is adjusted so that data movement and normalization control remain synchronized. Together, these changes make the optimized dual-core flow more stable and easier to integrate than the earlier version.

9 Sparsity-Aware Design

In Step 6, we implemented row-level sparsity (B2) in the normalization stage. For each query row, the design computes the accumulated row score

$$S_i = \sum_j |Q_i \cdot K_j|,$$

and compares it with a predefined threshold. If the row sum is below the threshold, that row is skipped and the normalized output is forced to zero; otherwise, the normal computation flow is preserved.

The threshold was selected from a Python reference model using the 30th percentile of the row sums. In hardware, the row sum is compared with this threshold to generate a registered skip signal. Fig. 11 shows the corresponding row-sum and skip-control waveform, and Fig. 12 shows the write-back results, where skipped rows are written as zero outputs. The hardware behavior matches the Python reference for the tested cases.

Compared with the baseline version used in Step 6, the B2 sparsity-aware design reduces the total power from 580.99 mW to 212.67 mW, which corresponds to about 63.4% reduction, as summarized in Table 4.

```
[[ 9 -101 -29 -73 8 -51 66 118]
 [ 62 -97 -64 -17 42 -53 54 121]
 [-10 -74 -29 -33 4 -32 18 96]
 [-23 28 -25 -165 -7 53 17 -16]
 [-24 -64 9 -59 -143 13 10 -13]
 [ 15 -19 -3 -42 38 -22 15 79]
 [-26 102 105 28 -16 -27 -31 -93]
 [-105 14 15 92 -34 38 -76 -83]]
[[-14 -25 29 21 -12 45 29 -59]
 [-32 29 23 61 5 -2 13 2]
 [ 1 -27 -26 6 -26 55 22 -35]
 [20 -83 23 -27 4 -9 18 -44]
 [ 23 -97 -30 -97 14 -31 66 13]
 [16 -24 -11 19 -9 74 19 -29]
 [41 -72 -34 -88 -21 32 -22 -27]
 [-8 24 -24 -38 -41 5 0 -43]]
SI_all: [455 510 296 334 335 233 428 457 234 167 198 228 371 201 337 183]
30% threshold: 230
Rows to skip (mask): [False False False False False False False False True True True
 False True False True]
Fraction skipped: 0.3125
```

Figure 10: Python threshold selection and skip mask.

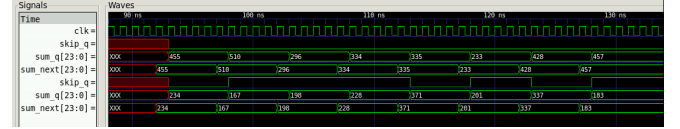


Figure 11: Row-sum and skip-control waveform.

```
##### sfp normalization and wb to pmem #####
Memory write to PSUM mem add 0 00003000250000a0001b0000200012000180002b
Memory write to PSUM mem add 0 0000500090000a0000700004000100000a00015
Memory write to PSUM mem add 1 0000000000000000000000000000000000000000
Memory write to PSUM mem add 1 000170002400018000060000f00014000140002d
Memory write to PSUM mem add 2 00005000260000f0001100002000100009000031
Memory write to PSUM mem add 2 0000000000000000000000000000000000000000
Memory write to PSUM mem add 3 0000000000000000000000000000000000000000
Memory write to PSUM mem add 3 0000a0000c0000b0004b00003000180000700007
Memory write to PSUM mem add 4 0000800017000030001500033000040000300004
Memory write to PSUM mem add 4 00008000230000a00023000050000b0001700004
Memory write to PSUM mem add 5 0000000000000000000000000000000000000000
Memory write to PSUM mem add 5 000080000b0000100018000160000c0000800002e
Memory write to PSUM mem add 6 0000800022000230000900005000090000a0001f
Memory write to PSUM mem add 6 0000d000180000b0001d000070000a0000700009
Memory write to PSUM mem add 7 0000000000000000000000000000000000000000
Memory write to PSUM mem add 7 0002a0000500006000240000d0000f0001e00021
```

Figure 12: Write-back results with skipped rows forced to zero.

Metric	Baseline	B2	Change
Total Power (mW)	580.99	212.67	-63.4%

Table 4: Power comparison between the baseline version and the B2 sparsity-aware design in Step 6.

10 Conclusion

In this project, we started from the provided single-core framework and gradually extended it to a dual-core design for the attention mechanism. Over the course of the work, we completed single-core RTL verification, output normalization, flat and hierarchical backend implementation, and dual-core integration across different clock domains. The final results show that the main design flow was completed successfully and that the key functional behavior was verified through simulation. We also explored several optimization and sparsity-aware directions as part of the later design stages.

Appendix

Total 101 references 119724.477440

Figure 13: Area report of the flat single-core design.

Total Power			

Total Internal Power:	43.60515735	72.2162%	
Total Switching Power:	15.32337566	25.3776%	
Total Leakage Power:	1.45287344	2.4062%	
Total Power:	60.38140602		

Figure 14: Power report of the flat single-core design.

Setup mode	all	reg2reg	default
WNS (ns):	-0.021	-0.021	0.139
TNS (ns):	-0.827	-0.827	0.000
Violating Paths:	83	83	0
All Paths:	14848	7678	10018

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 54.912%
(83.836% with Fillers)
Total number of glitch violations: 0

Figure 15: Timing report of the flat single-core design.

```

Begin Summary ...
Cells           : 0
SameNet        : 0
Wiring         : 0
Antenna        : 0
Short          : 0
Overlap        : 0
End Summary

```

Figure 16: DRC summary of the flat single-core design.

Baseline Backend Reports for the Sparsity-Aware Design

Report : reference
Design : core
Version: K-2015.06-SP2
Date : Sun Mar 15 13:56:59 2026

Attributes:
b - black box (unknown)
bo - allows boundary optimization
d - dont touch
mo - map only
h - hierarchical
n - noncombinational
r - removable
s - synthetic operator
u - contains unmapped logic

Reference	Library	Unit Area	Count	Total Area	Attributes

BUFFD1	tcbn65gpluswc	1.440000	16	23.040001	
KND8	tcbn65gpluswc	4.320000	1	4.320000	
INVD0	tcbn65gpluswc	1.080000	2	2.160000	
INVD1	tcbn65gpluswc	1.080000	2	2.160000	
INVD8	tcbn65gpluswc	4.320000	1	4.320000	
MUX2D0	tcbn65gpluswc	3.240000	224	725.760002	
NR2D0	tcbn65gpluswc	1.440000	2	2.880000	
NR2XD0	tcbn65gpluswc	1.440000	1	1.440000	
mac_array_col8_bw8_bw_psum20_pr8		46480.679664	1	46480.679664	h, n
ofifo_col8_bw20		17372.159918	1	17372.159918	h, n
sfp_row_col8_bw8		168135.122324	1	168135.122324	h, n
sram_w16_sram_bit64_0		12158.279878	1	12158.279878	h, n
sram_w16_sram_bit64_1		12156.119878	1	12156.119878	h, n
sram_w16_sram_bit160		30350.879140	1	30350.879140	h, n
Total 14 references				287419.320807	

Figure 17: Post-route area summary of the hierarchy single-core design.

```

Path 1: VIOLATED Hold Check with Pin sfp_instance/sfp_out_sign2_reg_1/CP
Endpoint: sfp_instance/sfp_out_sign2_reg_1/D (v) checked with
leading edge of 'clk'
Beginpoint: sfp_instance/fifo_inst_int/rd_ptr_reg_3/Q (^) triggered by
leading edge of 'clk'
Path Groups: {clk}
Analysis View: BC_VIEW
Other End Arrival Time 1.259
+ Hold 0.008
+ Phase Shift 0.000
- CPPR Adjustment 0.000
= Required Time 1.267
Arrival Time 0.436
Slack Time -0.831
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.095
= Beginpoint Arrival Time 0.095

```

Figure 18: Post-route timing summary of the hierarchy single-core design.

```

* Power Units = 1mW
*
* Time Units = 1e-09 secs
*
* report_power -outfile core.post_route.power.rpt
*
-----
Total Power
-----
Total Internal Power: 372.78488832 54.5078%
Total Switching Power: 300.69913126 43.9676%
Total Leakage Power: 10.42696084 1.5246%
Total Power: 683.91098061

```

Figure 19: Post-route power summary of the hierarchy single-core design.

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	0.2375	463.7914	8.1151e+04	464.1045	(79.88%)	
register	108.5763	0.8092	9.5093e+05	110.3369	(18.99%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	3.5861	1.9158	1.0457e+06	6.5477	(1.13%)	
Total	112.4000 mW	466.5165 mW	2.0778e+06 nW	580.9890 mW		

Figure 20: Post-route power report of the dual-core design.

clock clk (rise edge)	1.000	1.000
clock network delay (ideal)	0.000	1.000
core_instance0/ofifo_inst/col_idx_4_fifo_instance/q3_reg_9/_CP (EDF0D1)	0.000	1.000 r
library setup time	-0.095	0.905
data required time		0.905
data required time		0.905
data arrival time		-0.975
slack (VIOLATED)		-0.070

Figure 21: Post-route timing report of the dual-core design.

Reference	Library	Unit Area	Count	Total Area	Attributes
cdc_4phase		0.000000	2	0.000000	b
core_col8_bw8_bw_psum20_pr16_0		141685.559039			
			1	141685.559039	b, h, n
core_col8_bw8_bw_psum20_pr16_1		141723.359053			
			1	141723.359053	b, h, n
Total 3 references				283408.918092	

Figure 22: Area report of the dual-core design.

```
#Post Route via swapping is done.
#Total number of nets with non-default rule or having extra spacing = 74
#Total wire length = 1083083 um.
#Total half perimeter of net bounding box = 903036 um.
#Total wire length on LAYER M1 = 17406 um.
#Total wire length on LAYER M2 = 422319 um.
#Total wire length on LAYER M3 = 452077 um.
#Total wire length on LAYER M4 = 166058 um.
#Total wire length on LAYER M5 = 15696 um.
#Total wire length on LAYER M6 = 5487 um.
#Total wire length on LAYER M7 = 4039 um.
#Total wire length on LAYER M8 = 0 um.
#Total number of vias = 476844
#Total number of multi-cut vias = 308664 ( 64.7%)
#Total number of single cut vias = 168180 ( 35.3%)
#Up-Via Summary (total 476844):
#-----
#              single-cut          multi-cut          Total
#-----
# Metal 1      166636 ( 64.4%)      92062 ( 35.6%)      258698
# Metal 2      1506 ( 0.8%)          186911 ( 99.2%)      188417
# Metal 3       34 ( 0.1%)          27855 ( 99.9%)      27889
# Metal 4       1 ( 0.1%)          1246 ( 99.9%)       1247
# Metal 5       2 ( 0.5%)           366 ( 99.5%)        368
# Metal 6       1 ( 0.4%)           224 ( 99.6%)        225
#-----
#              168180 ( 35.3%)      308664 ( 64.7%)      476844
```

Figure 23: Post-route metal and via summary of the dual-core design.

```
VERIFY DRC ..... Sub-Area : 56 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 57 of 66
VERIFY DRC ..... Sub-Area : 57 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 58 of 66
VERIFY DRC ..... Sub-Area : 58 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 59 of 66
VERIFY DRC ..... Sub-Area : 59 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 60 of 66
VERIFY DRC ..... Sub-Area : 60 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 61 of 66
VERIFY DRC ..... Sub-Area : 61 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 62 of 66
VERIFY DRC ..... Sub-Area : 62 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 63 of 66
VERIFY DRC ..... Sub-Area : 63 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 64 of 66
VERIFY DRC ..... Sub-Area : 64 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 65 of 66
VERIFY DRC ..... Sub-Area : 65 complete 0 Viols.
VERIFY DRC ..... Sub-Area : 66 of 66
VERIFY DRC ..... Sub-Area : 66 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:56.3 ELAPSED TIME: 57.00 MEM: 84.7M) ***
```

Figure 24: DRC summary of the dual-core design.

XOR2D2	tcbn65gpluswc	3.960000	43	170.280002
XOR3D0	tcbn65gpluswc	6.120000	16	97.919998
cdc_4phase		0.000000	2	0.000000 b
Total 217 references				591183.718465

Figure 25: Baseline area report used for the sparsity-aware design.

```
Global Operating Voltage = 0.9
Power-specific unit information :
Voltage Units = 1V
Capacitance Units = 1.000000pf
Time Units = ns
Dynamic Power Units = 1mW (derived from V,C,T units)
Leakage Power Units = 1nW

Cell Internal Power = 71.7491 mW (21%)
Net Switching Power = 266.7010 mW (79%)
-----
Total Dynamic Power = 338.4502 mW (100%)
Cell Leakage Power = 4.3651 mW
```

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	6.2589	136.0823	1.9433e+06	144.3083	(67.08%)	
register	64.0544	0.2331	1.4740e+06	65.7615	(30.84%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	1.4355	0.7570	9.4788e+05	3.1403	(1.47%)	
Total	71.7487 mW	137.0724 mW	4.3651e+06 nW	213.2102 mW		

Figure 26: Baseline power report used for the sparsity-aware design.

clock clk (rise edge)	2.500	2.500
clock network delay (ideal)	0.000	2.500
core_instance1/sfp_instance_sfp_out_sign4_reg_0/_CP (EDF0D1)	0.000	2.500 r
library setup time	-0.091	2.409
data required time		2.409
data arrival time		-5.515
slack (VIOLATED)		-3.107

Figure 27: Baseline timing report used for the sparsity-aware design.

```
Cell Internal Power = 127.7569 mW (61%)
Net Switching Power = 83.0768 mW (39%)
-----
Total Dynamic Power = 210.8337 mW (100%)
Cell Leakage Power = 1.8373 mW
```

Figure 28: B2 timing report used for the sparsity-aware design.